
AGENDA 8

**Introdução à Análise
de Sistemas e
Projetos.**

SEAD- Superintendência de Ensino e Pesquisa nas
modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO
EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO
CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS
TECNOLOGIA DA INFORMAÇÃO I

Expediente

Autores:

José Mendes da Silva Neto

Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2026



MOMENTO REFLEXÃO



Lucas, recém-contratado como analista de sistemas, entrou na empresa animado para participar do seu primeiro projeto de verdade. Mal sabia ele que, logo no início, seria envolvido em uma discussão acalorada entre os colegas. Palavras como "cascata", "incremental" e "ágil" surgiam a todo momento, mas para Lucas, tudo aquilo parecia um novo idioma. Sem entender exatamente o que significavam esses termos, ele percebeu que a escolha entre essas abordagens era motivo de dúvidas e debates constantes na equipe.



Imagem 1: Comunicação no Ambiente de Tecnologia.

Lucas se perguntou: será que existe uma forma certa de planejar o desenvolvimento de um sistema? Ou cada projeto pede um jeito diferente de organizar o trabalho?

Enquanto observava os colegas defendendo diferentes ideias, Lucas sentiu que estava prestes a descobrir algo fundamental para seu crescimento profissional — mas, por enquanto, a grande questão era: o que, afinal, é um modelo de desenvolvimento de software e por que isso faz tanta diferença na prática?



PORQUE APRENDER?

Você já percebeu como o sucesso de um sistema começa muito antes da primeira linha de código?

Quando você entende a fundo os modelos de desenvolvimento de software, passa a enxergar o planejamento como um diferencial estratégico na sua atuação. Não se trata apenas de executar tarefas, mas de compreender a lógica por trás das escolhas que vão orientar todo o projeto.

Ao dominar esses modelos, você desenvolverá a capacidade de escolher a abordagem mais adequada para cada

Imagem 2: Modelos de Desenvolvimento.



tipo de desafio, organizando o projeto e a equipe de forma eficiente. Isso significa saber quando seguir um planejamento detalhado ou quando apostar em métodos mais flexíveis, sempre com o objetivo de atender de verdade as necessidades do usuário.

Você passará a perceber como uma decisão bem fundamentada pode evitar retrabalho, reduzir custos e garantir entregas de maior qualidade.

Mais do que isso, será capaz de facilitar a comunicação entre todos os envolvidos, antecipar riscos e adaptar-se rapidamente às mudanças que surgem no caminho.

Com esse domínio, você se torna um profissional preparado para planejar, organizar e conduzir projetos de sistemas que realmente fazem a diferença — seja em grandes empresas, start-ups ou nos seus próprios projetos. O mercado valoriza quem consegue transformar complexidade em resultado concreto, e é exatamente esse ganho que você leva ao aprender profundamente sobre modelos de desenvolvimento de software.

PARA COMEÇAR O ASSUNTO



Imagem 3: Apresentação modelos de desenvolvimento.

Lucas foi chamado para participar de sua primeira reunião de equipe. A empresa, especializada em desenvolver sistemas para o comércio local, estava prestes a iniciar um projeto importante para um cliente exigente. Durante a reunião, o gerente apresentou os desafios do projeto e pediu sugestões sobre como planejar as próximas etapas. Sem muita experiência, Lucas percebeu que cada colega defendia uma forma diferente de organizar o trabalho: alguns preferiam

planejar tudo nos mínimos detalhes antes de começar, outros sugeriam avançar por partes, entregando pequenas soluções a cada etapa, enquanto havia quem defendesse mudanças rápidas e adaptação constante. Para Lucas, tudo era novidade: ele ainda tentava entender o que significavam termos como “modelo cascata”, “incremental” e “ágil”.

Foi então que as perguntas começaram a surgir:

- Qual é a diferença prática entre essas formas de organizar o desenvolvimento de um sistema?
- Como saber quando é melhor planejar tudo antes ou adaptar no caminho?
- Será que existe um modelo ideal para todo tipo de projeto, ou cada situação pede uma escolha diferente?
- O que pode acontecer se a equipe não souber escolher bem essa abordagem?

Essas dúvidas mostram que, muitas vezes, o problema não está apenas na execução, mas na forma como o trabalho é planejado e gerenciado desde o início. Para entender melhor esse cenário e descobrir caminhos para organizar projetos de sistemas com mais eficiência, vamos mergulhar nos principais modelos de desenvolvimento e suas aplicações no mundo real.



MERGULHANDO NO TEMA

Imagine que você acaba de receber a missão de criar uma solução tecnológica para facilitar o dia a dia de uma pequena empresa. Antes de pensar em programar ou escolher ferramentas, surge uma pergunta essencial: como transformar uma necessidade real em um sistema eficiente? É aí que entra a análise de sistemas, um campo fundamental do desenvolvimento de software responsável por entender, planejar e estruturar soluções que realmente façam diferença.

No passado, muitos projetos de sistemas falhavam porque começavam direto na execução, sem um estudo aprofundado do problema ou das necessidades dos usuários. Com o tempo, percebeu-se que o verdadeiro sucesso de um sistema depende de uma etapa anterior: analisar cuidadosamente o contexto, os processos envolvidos e as expectativas de todos os envolvidos.

Análise de sistemas é o processo de investigar, compreender e definir como um sistema deve funcionar para resolver problemas específicos de uma organização ou usuário. Isso envolve coletar informações, identificar necessidades, mapear processos e propor soluções viáveis — sempre alinhadas aos objetivos do projeto.

Na prática, a análise de sistemas exige do profissional uma postura investigativa e colaborativa. Você precisa ouvir diferentes pessoas, conectar informações dispersas e transformar tudo isso em requisitos claros para o desenvolvimento. É como montar um quebra-cabeça, onde cada peça — seja um processo, uma limitação ou uma expectativa — precisa se encaixar para que o resultado final seja funcional e relevante.

Como destaca Sommerville (2000), a análise de sistemas é uma etapa crítica que determina o sucesso ou fracasso do projeto, pois dela depende a compreensão correta do que realmente precisa ser desenvolvido.

Compreender o que é análise de sistemas é o primeiro passo para atuar de forma estratégica no desenvolvimento de soluções tecnológicas.

Você pode usar a inteligência artificial para ampliar sua compreensão.

Experimente utilizar o seguinte prompt:

"Explique, de forma simples, o que é análise de sistemas e por que ela é essencial no desenvolvimento de software. Dê um exemplo de problema que pode surgir quando essa etapa é ignorada."

Após ler a resposta, reflita:

O que chamou sua atenção na explicação da IA?

O exemplo faz sentido para sua futura atuação profissional?

Algo ficou diferente do que você pensava?

Lembre-se: a IA serve como apoio, mas sua análise crítica é fundamental.

Agora que você já refletiu sobre o papel fundamental da análise de sistemas e também conferiu como a inteligência artificial pode apresentar diferentes perspectivas sobre o tema é hora de consolidar o aprendizado com base no material didático.

Ao comparar sua própria reflexão, a resposta da IA e os conceitos apresentados aqui, você desenvolve um olhar mais crítico e autônomo para atuar na área de sistemas.

Retomando o caso de Lucas, pense em como a análise de sistemas faz diferença antes mesmo do desenvolvimento começar: ela permite identificar necessidades reais, evitar retrabalho e alinhar expectativas entre a equipe e o cliente. O conceito envolve desde a investigação inicial até a definição clara de requisitos, tornando-se uma etapa estratégica para o sucesso de qualquer projeto. Se você utilizou a inteligência artificial, provavelmente percebeu que, sem uma boa análise de sistemas, problemas como funcionalidades desnecessárias, atrasos e insatisfação do cliente podem surgir facilmente. É por isso que entender esse conceito é essencial para quem quer se destacar no mercado de tecnologia.

Modelos de Desenvolvimento de Sistemas

Imagine que você precisa construir uma casa. Antes de começar, é preciso definir o que será feito em cada etapa, quem faz o quê e como tudo será acompanhado. No desenvolvimento de sistemas, acontece algo parecido: os modelos de desenvolvimento são conjuntos de práticas e etapas organizadas para planejar, construir, testar e entregar um sistema, desde a ideia inicial até a manutenção. Esses modelos servem para dar ordem ao processo, evitar improvisos e garantir que o produto final realmente atenda às necessidades de quem

vai usar. Cada modelo traz uma lógica própria: alguns valorizam o planejamento detalhado antes de começar, outros preferem ciclos rápidos de entrega e adaptação. O importante é entender que a escolha do modelo influencia diretamente o sucesso do projeto, a comunicação da equipe e a satisfação do cliente.

Conhecer os principais modelos de desenvolvimento permite que você, como profissional, saiba escolher a abordagem mais adequada para cada situação evitando retrabalho, desperdício de tempo e recursos, e aumentando suas chances de entregar um sistema que realmente faz a diferença.

Modelos de Desenvolvimento de Sistemas

Antes de existirem métodos mais flexíveis, o desenvolvimento de sistemas seguia modelos conhecidos como tradicionais. Esses modelos foram criados para trazer organização e previsibilidade ao processo, especialmente em projetos de maior porte, onde cada etapa precisava ser planejada com antecedência e executada em sequência.

A lógica dos modelos tradicionais é simples: tudo começa pelo levantamento detalhado dos requisitos, seguido pelo projeto, implementação, testes e, finalmente, a entrega e manutenção. Cada fase só se inicia quando a anterior está completamente concluída, o que ajuda a evitar imprevistos, mas pode dificultar adaptações ao longo do caminho.

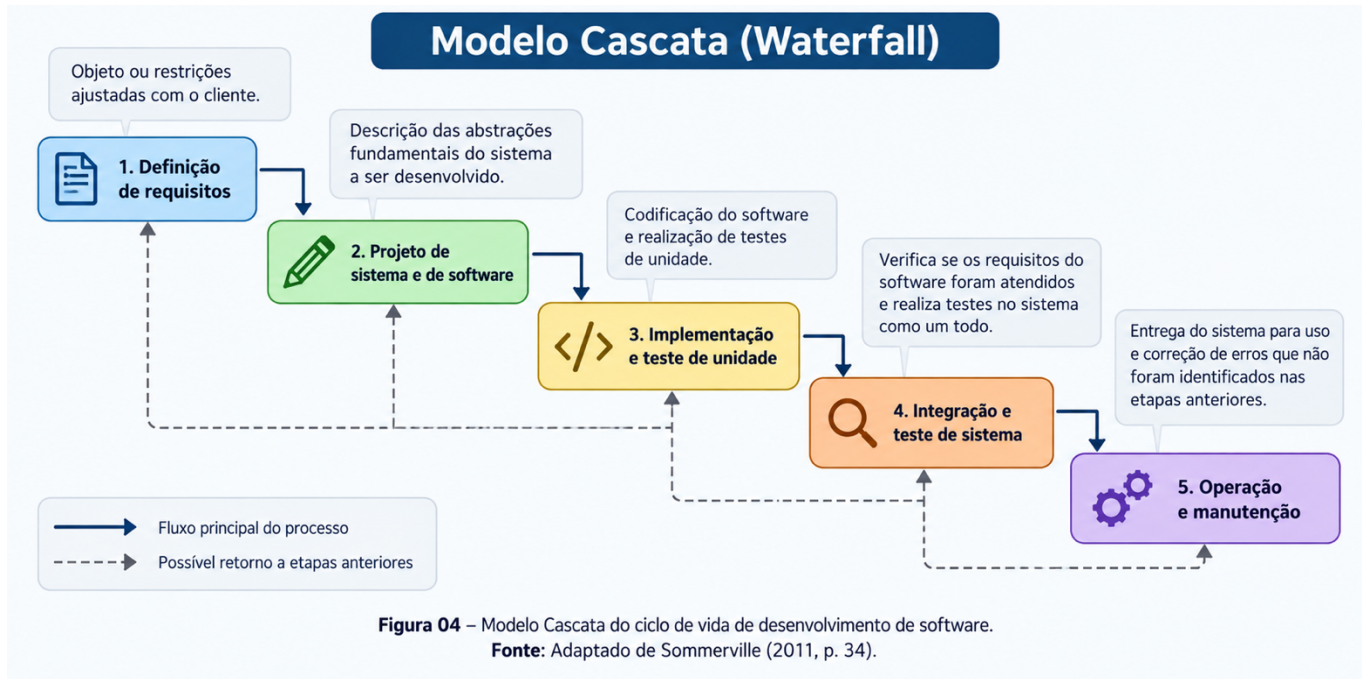
Essas metodologias valorizam a documentação rigorosa, o planejamento detalhado e o controle de cada etapa, tornando o processo mais seguro em ambientes onde as mudanças são raras ou indesejadas. São escolhas comuns em projetos em que as necessidades já estão claras e o contexto não muda com frequência.

Modelo em Cascata ou Clássico

Imagine que você está participando de um projeto em que cada etapa precisa ser concluída e aprovada antes de passar para a próxima, como se fosse montar um quebra-cabeça seguindo uma ordem rigorosa. O Modelo Cascata funciona exatamente assim: é uma abordagem sequencial, onde o desenvolvimento do sistema acontece em fases bem definidas, cada uma com seu objetivo claro e documentação detalhada.

No contexto do desenvolvimento de software, o Modelo Cascata foi um dos primeiros métodos estruturados a ganhar popularidade, especialmente em projetos nos quais as exigências eram estáveis e bem compreendidas desde o início. Ele se organiza em etapas como: análise de requisitos, projeto, implementação, testes, e, por fim, operação e manutenção. Só é possível avançar para a próxima etapa quando a anterior estiver completamente finalizada e validada.

O Modelo Cascata é o primeiro processo publicado (ROYCE, 1970), que também é conhecido por abordagem top down, foi o mais utilizado nos anos 80, onde exigia uma complementação a cada fase. O termo “cascata” vem da proposta de desenvolvimento de projeto encadeada, sequencial e sistemática, que pressupõe terminar uma etapa para iniciar outra.



Esse modelo considera cinco etapas mais comuns:

Etapa 1 - Análise e definição de requisitos: devemos entender quais as funcionalidades que o sistema irá executar. Neste processo, é necessário definir os requisitos do sistema e validar os serviços e restrições.

Etapa 2 - Projeto de sistema e software: compõe atividades para assegurar que o sistema ou produto de software seja desenvolvido e entregue ao cliente no prazo estipulado.

Etapa 3 - Implementação e teste de unidade: a implementação ocorre após o projeto da solução. A homologação ou aceitação acontece posteriormente, depois do desenvolvimento, da integração e dos testes.

Etapa 4 - Integração e teste de sistema: verifica se foram solucionados erros de “comportamento” do software e assegura que as entradas definidas produzam resultados reais que coincidam com os requisitos especificados pelo cliente.

Etapa 5 - Operação e manutenção: garante o bom funcionamento dos serviços direcionando-o para um bom desempenho.

Apesar do resultado de cada fase ser uma aprovação para o início da fase seguinte, como mostra a imagem 04, na prática das empresas, esse modelo não funcionava de forma tão linear como se esperava, pois, por vezes, acontecia encontrar problemas relacionados à fase anterior, o que causava retrabalho e atraso na fase atual (PUC https://www.maxwell.vrac.puc-rio.br/21839/21839_3.PDF).

“Devido aos custos de produção e aprovação de documentos, as iterações são onerosas e envolvem um ‘retrabalho’ significativo. Portanto, após um pequeno número de iterações, é normal suspender partes do desenvolvimento, como a especificação e prosseguir com os estágios posteriores do desenvolvimento. Os problemas são resolvidos em um outro momento, ignorados ou reprogramados. O congelamento prematuro de requisitos pode significar que o sistema não fará o que o usuário deseja. Isso pode também levar a sistemas mal estruturados, pois os problemas de projeto foram contornados por meio de artifícios de implementação.”

(SOMMERVILLE, 2007).

Assim, a grande desvantagem do Modelo Cascata é com relação à mudança necessária para aprimoramento do projeto tornando-o engessado, ou seja, impossibilita atender novas mudanças.

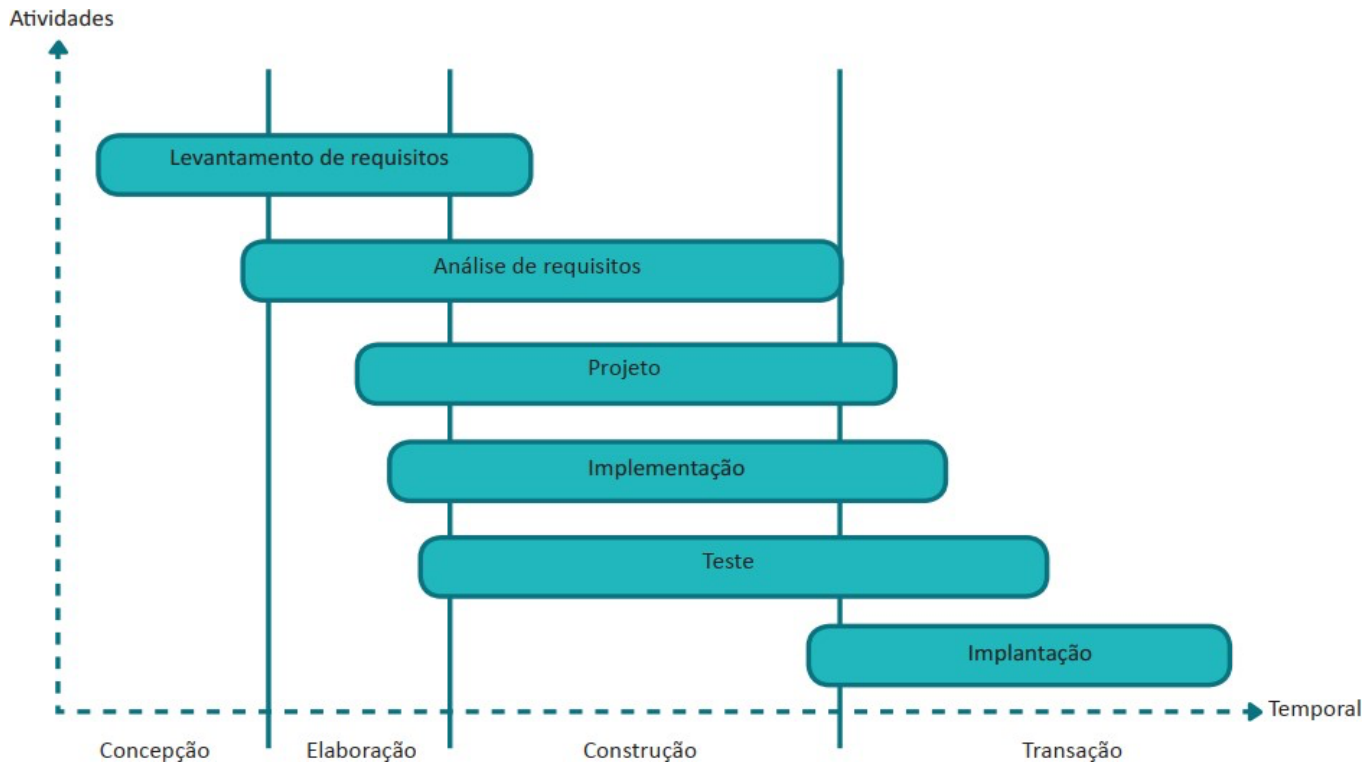
Por outro lado, uma das maiores vantagens desse modelo é a documentação abrangente e detalhada, realizada em cada fase do projeto, o que possibilita maior monitoramento do processo, de acordo com as especificações do cliente.

Em resumo, o Modelo Cascata é ideal para projetos em que as necessidades são claras desde o início e as mudanças são mínimas. Entender como ele funciona é essencial para saber quando aplicá-lo ou buscar métodos mais flexíveis.

Modelo Iterativo e Incremental

No ambiente de desenvolvimento de sistemas, nem sempre é possível ter todas as respostas logo no início do projeto. Muitas vezes, os requisitos vão mudando, novas necessidades surgem e o cliente só percebe o que realmente precisa depois de ver o sistema funcionando. Para lidar com esse cenário, surgiram os modelos incremental e iterativo, que propõem uma abordagem mais flexível e adaptável.

Diferente dos modelos tradicionais, aqui o sistema é desenvolvido em partes menores, chamadas de incrementos. A cada ciclo, uma nova funcionalidade é planejada, implementada e testada. Isso permite entregar versões parciais do sistema já em funcionamento, coletar feedback dos usuários e ajustar o que for necessário antes de avançar para a próxima etapa.



O modelo incremental foca em construir o sistema por blocos: a cada entrega, o produto ganha novas funções e se aproxima do objetivo final. Já o modelo iterativo destaca a repetição de ciclos, onde cada versão é aprimorada com base em revisões e testes constantes. Muitas vezes, os dois conceitos aparecem juntos, pois na prática é comum construir incrementos em ciclos de iteração. Essa lógica oferece vantagens como maior flexibilidade para mudanças, identificação precoce de problemas e engajamento contínuo do cliente no processo. Porém, exige organização e comunicação eficiente entre todos os envolvidos, já que várias atividades podem ocorrer simultaneamente e os requisitos podem evoluir ao longo do tempo.

Segundo Sommerville (2000), a abordagem incremental e iterativa reduz riscos e custos ao tornar o desenvolvimento mais próximo da realidade do usuário e mais preparado para lidar com mudanças inesperadas.

No mercado atual, compreender esses modelos é essencial para atuar em projetos dinâmicos, onde a capacidade de adaptação faz toda a diferença para o sucesso do sistema.

Modelo Espiral

Diferentemente do modelo Iterativo e Incremental, o modelo espiral utiliza a metodologia de desenvolvimento em sequência de interações e a cada volta representa uma fase (atividade).

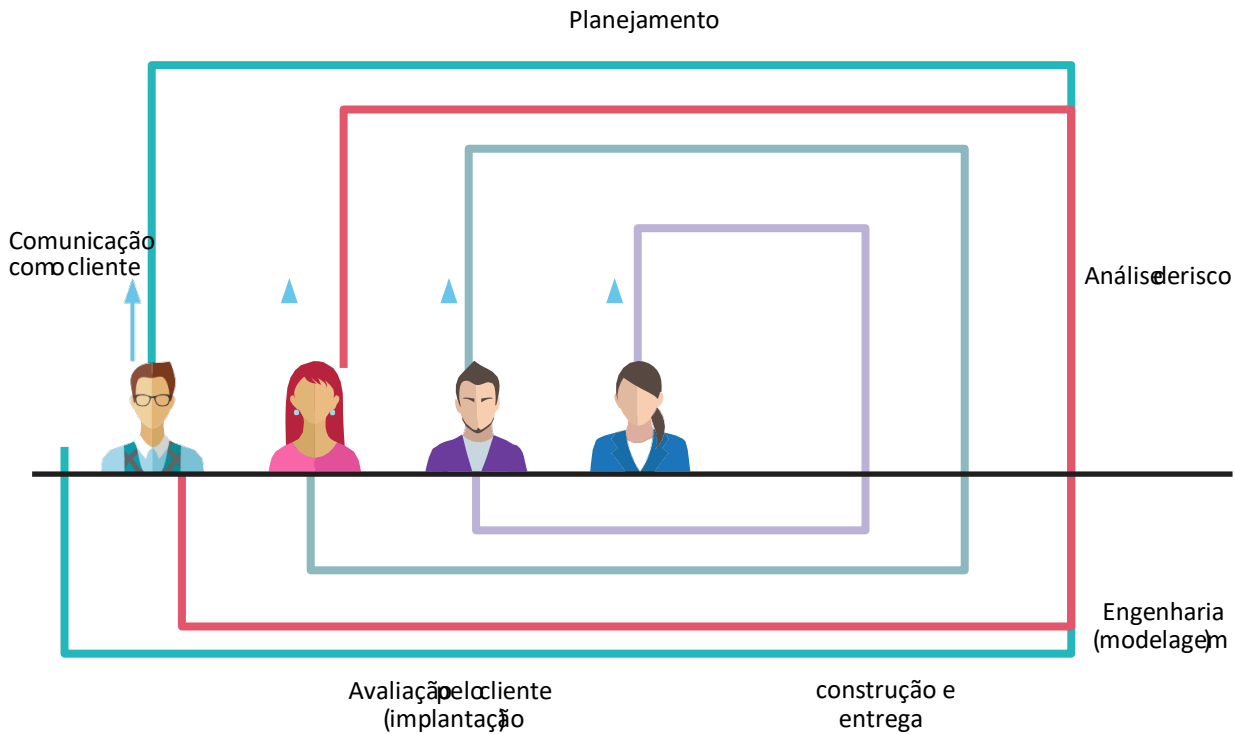


Imagem 06 - Arquivo SEAD

O modelo espiral, criado por Barry Boehm em 1988, permite que as ideias e a inovação sejam verificadas e avaliadas constantemente. Isso porque, a cada interação, existe a volta da espiral que pode ser baseada em um modelo diferente e ter diferentes atividades. Esse padrão caracteriza-se, portanto, pelo desenvolvimento em sequência, aumentando a complexidade do processo conforme se chega mais próximo do produto final (PIVA; OLIVEIRA, 2010).

A cada ciclo da espiral, uma série de atividades é realizada em uma determinada ordem, como comunicação com o cliente, planejamento, análise de riscos, Modelagem, Construção e Entrega e Avaliação pelo Cliente.

Observamos que as metodologias tradicionais funcionavam de maneira muito limitada. As tarefas seguiam uma ordem linear de desenvolvimento, fazendo com que se fizesse necessário atender a todos os objetivos antes de iniciar o desenvolvimento do projeto. Cada uma das fases somente poderia ser iniciada depois que a tarefa anterior fosse finalizada, dessa forma, o que já estava pronto era difícil de ser verificado enquanto as outras etapas estivessem em andamento. Essas metodologias traziam alguns problemas, como:

- Perda de tempo no processo de planejamento e atraso no desenvolvimento;
- Dificuldade em antecipar as necessidades futuras;
- Atraso na entrega do produto;
- Erros apenas percebidos quando o produto final fosse entregue ao cliente.

Abordagens Ágeis

À medida que o mercado de tecnologia evoluiu, ficou claro que projetos de sistemas precisam responder rapidamente a mudanças e valorizar a colaboração entre pessoas. Foi desse contexto que surgiram os modelos ágeis, baseados em princípios que colocam a flexibilidade, o trabalho em equipe e a entrega contínua de valor para o cliente como prioridades.

Os modelos ágeis propõem ciclos curtos de desenvolvimento, chamados de iterações, nos quais o sistema é construído, revisado e aprimorado de forma contínua. A cada ciclo, o time entrega uma parte funcional do produto e, junto com o cliente, avalia o que deve ser ajustado ou priorizado em seguida. Isso facilita adaptações rápidas, evita desperdício de esforço e favorece que o resultado final esteja mais alinhado com as necessidades reais do usuário.

O Manifesto Ágil, criado em 2001 por especialistas da área, sintetiza essa filosofia em quatro valores centrais:

- indivíduos e interações mais que processos e ferramentas;
- software em funcionamento mais que documentação abrangente;
- colaboração com o cliente mais que negociação de contratos;
- responder a mudanças mais que seguir um plano.

Esses valores orientam práticas como reuniões frequentes, comunicação aberta, entregas frequentes e foco constante na satisfação do cliente.

Ao adotar modelos ágeis, você aprende a trabalhar em equipe, a lidar com incertezas e a adaptar o projeto conforme novas demandas aparecem. Mais do que seguir etapas rígidas, o profissional ágil está preparado para buscar soluções criativas e eficientes em cenários dinâmicos.

Conhecer os fundamentos dos modelos ágeis é fundamental para quem deseja atuar em equipes modernas de desenvolvimento de sistemas, onde a capacidade de adaptação e a comunicação são tão importantes quanto o domínio técnico.

Métodos Ágeis

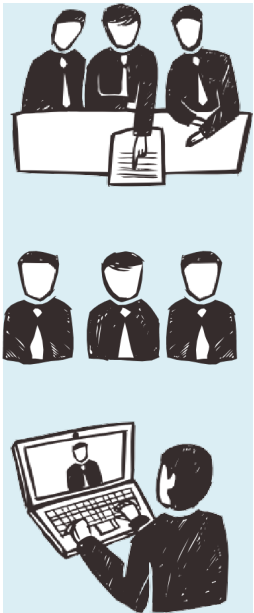
A metodologia Scrum tem como objetivo estruturar o trabalho em equipe no planejamento e desenvolvimento de softwares. Os projetos são divididos em ciclos (tipicamente mensais) chamados de Sprints, com times multifuncionais e auto gerenciáveis. A estrutura do Scrum procura aproveitar o que a equipe tem de melhor em relação à otimização de tempo, rapidez e qualidade do trabalho.

A essência dessa metodologia baseia-se na ideia de verificar de tempos em tempos se o projeto está no caminho correto e se o que está sendo desenvolvido está de acordo com a expectativa do cliente. Essa ação é

chamada de “transparência, inspeção e adaptação”.

Todos os dias são executados um conjunto de atividades do projeto, por exemplo: Em sua casa você tem uma lista de afazeres, como: ir ao supermercado, levar as crianças à escola, organizar suas mochilas para certificar-se de que o material da aula está em ordem etc. Todas essas atividades devem ser priorizadas e cumpridas diariamente, assim são os projetos com suas atividades pontuais.

Vamos conhecer a composição da equipe:



O **Product Owner** é o responsável por maximizar o valor do produto e pelo gerenciamento eficaz do Product Backlog, incluindo sua ordenação. Ele pode representar necessidades de vários stakeholders. No Sprint Planning, por meio de discussão com o Product Owner, os Developers selecionam os itens do Product Backlog que entrarão na Sprint.

Team é a equipe que irá desenvolver o projeto, com auxílio do Product Owner. A equipe deve conter no mínimo 6 pessoas e no máximo 10, organizadas, autogerenciáveis e multifuncionais. É a equipe que irá definir o que irão realizar em cada sprint, definindo a melhor estratégia para atingir o sucesso do projeto.

Scrum Master é responsável pelo processo Scrum. Auxilia o Team e assegura que a equipe não se comprometa excessivamente com relação àquilo que é capaz de realizar durante um Sprint.

Agora que sabemos como é composta a equipe, vamos entender como funciona as Sprints.

Sprints são interações de um ciclo de desenvolvimento em Scrum. Cada uma recebe um incremento do produto, conforme as entregas da equipe. O tempo para cada sprint pode ocorrer em uma semana e terminar no máximo em um mês e nesse período terá que implementar as funções no Sprint Backlog a qual chamamos de histórias. Na figura 07, podemos observar como funciona este processo.

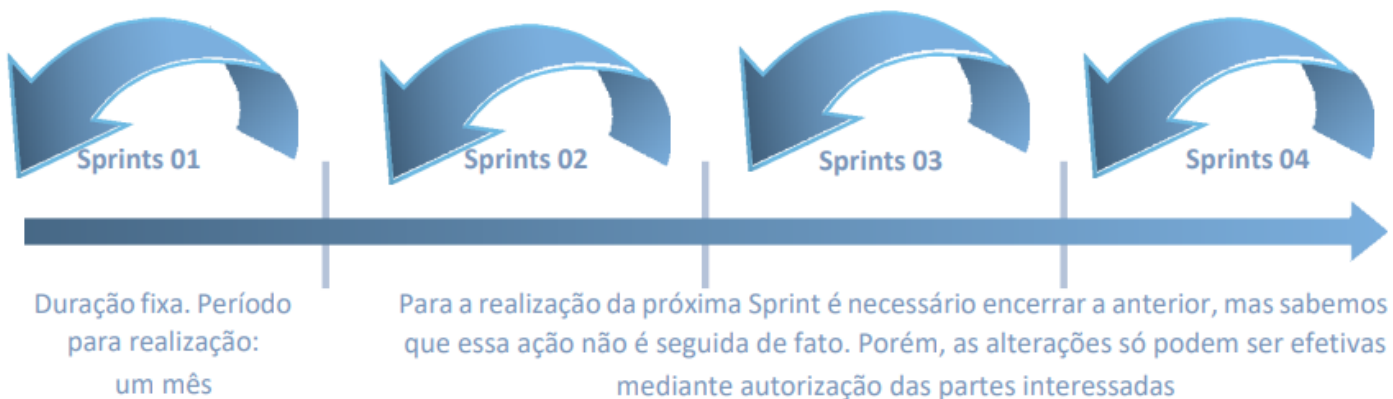


Figura 07 - Arquivo SEAD

Sprint Backlog é a lista de atividades relacionadas pelo time Scrum, e devem ser concluídas no período de

cada sprint. Na reunião de planejamento, a equipe relaciona as tarefas e o tempo necessário para execução e o prazo de término das atividades. Para o acompanhamento das tarefas é feita uma tabela com as sprints backlog e o responsável pelas ações. Uma vez por dia, é necessária a atualização da planilha, para que o Scrum Master gere um gráfico com as atividades realizadas e pendentes.

Sprint Planning

A Sprint Planning (Planejamento da Sprint) é o evento que inicia uma Sprint. Nesse momento, todo o **Scrum Team** trabalha de forma colaborativa para definir o que será realizado durante a Sprint e estabelecer um plano para alcançar a Meta da Sprint (Sprint Goal). Durante a Sprint Planning, três questões principais orientam o planejamento:

Por que esta Sprint é valiosa?

O Scrum Team define a Meta da Sprint, indicando o valor que se pretende alcançar naquele período.

O que pode ser realizado nesta Sprint?

Os Developers, em discussão com o Product Owner, selecionam os itens do Product Backlog que poderão ser desenvolvidos durante a Sprint.

Como o trabalho será realizado?

Os Developers planejam como transformar os itens selecionados em um Incremento que atenda à Definição de Pronto (Definition of Done).

A Meta da Sprint, os itens do Product Backlog selecionados e o plano para realizá-los formam o Sprint Backlog.

Daily Scrum

Após o planejamento, durante a Sprint ocorre diariamente a Daily Scrum, um evento de 15 minutos destinado aos Developers. Seu objetivo é verificar o progresso em direção à Meta da Sprint e adaptar o Sprint Backlog quando necessário, organizando o trabalho para o próximo dia.

Os Developers podem escolher a estrutura e as técnicas que considerarem mais adequadas para conduzir a Daily Scrum. Uma dinâmica bastante conhecida consiste em discutir três perguntas:

- O que fiz desde a última reunião para contribuir com a Meta da Sprint?
- O que pretendo fazer hoje para contribuir com a Meta da Sprint?
- Existe algum impedimento que possa dificultar o andamento do trabalho?

Importante: essas três perguntas são apenas uma possibilidade de organização da Daily Scrum. O Scrum Guide atual não determina que elas sejam utilizadas. O essencial é que a reunião permita aos Developers avaliar o progresso em direção à Meta da Sprint e produzir um plano de trabalho para o próximo dia.

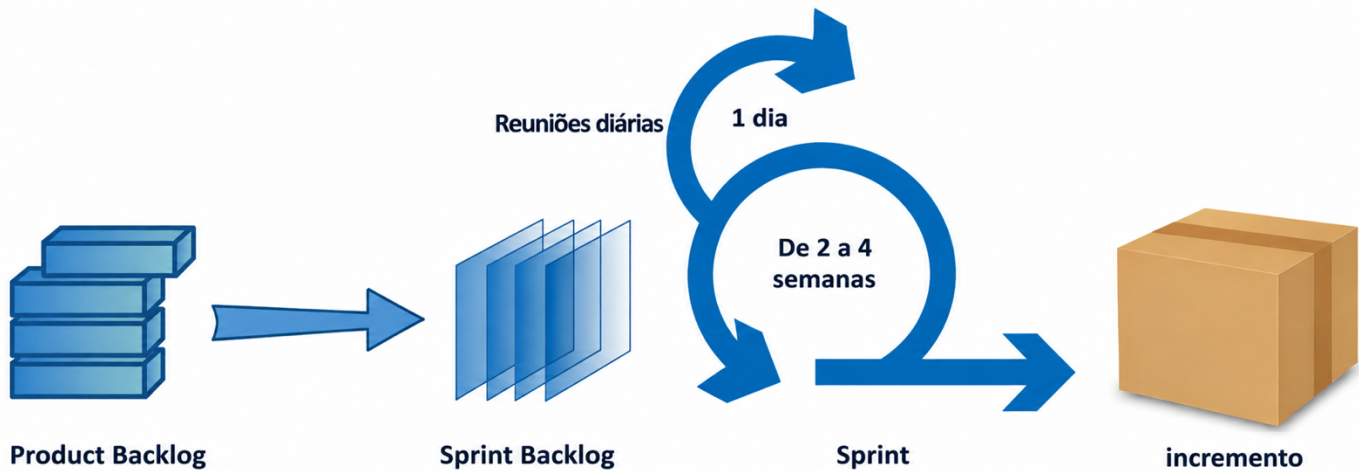


Imagem 08-Desenvolvimento Ágil utilizando Scrum

VOCÊ NO COMANDO

Jonas, técnico em Desenvolvimento de Sistemas, decidiu criar um controle das vendas para seu pai que estava tendo problemas para calcular com precisão o seu lucro e o total das despesas diárias. Vendo a necessidade do seu pai, Jonas decidiu adiantar o seu projeto, eliminando a etapa de validação do software.

Quais são as possíveis consequências que ele poderá enfrentar por ter excluído essa etapa?

Veja se você conseguiu resolver os desafios:

A etapa de validação é fundamental em qualquer modelo de desenvolvimento de software, pois garante que o sistema funcione corretamente e atenda às necessidades do usuário. Ao pular essa etapa, Jonas corre o risco de entregar um sistema com erros de funcionamento. Isso pode gerar informações incorretas e prejudicar a tomada de decisões no negócio de seu pai. Além disso, falhas não identificadas previamente podem causar retrabalho e exigir correções após o sistema já estar em uso. Outro impacto possível é a perda de confiança do usuário no sistema desenvolvido. Dessa forma, a validação é essencial para garantir a qualidade, a confiabilidade e a eficiência do software.

Glossário Técnico

- **Análise de Sistemas:** Processo de estudar e definir as necessidades de um sistema, identificando requisitos e propondo soluções para problemas organizacionais ou de negócio.
- **Modelo Cascata:** Metodologia tradicional de desenvolvimento de sistemas em que as etapas são realizadas de forma sequencial e cada fase só começa após a anterior ser finalizada.
- **Incremento:** Parte funcional de um sistema entregue em cada ciclo de desenvolvimento, permitindo evolução progressiva do produto final.
- **Iteração:** Repetição de um ciclo de atividades no desenvolvimento de sistemas, com cada nova versão sendo aprimorada a partir de revisões e testes.
- **Modelo Ágil:** Conjunto de métodos de desenvolvimento de software que priorizam flexibilidade, colaboração e entregas rápidas, adaptando-se facilmente às mudanças de requisitos.
- **Manifesto Ágil:** Documento criado em 2001 que define os valores e princípios fundamentais dos métodos ágeis de desenvolvimento de software.
- **Requisito:** Necessidade ou condição que um sistema deve atender, identificada durante a análise e registrada para orientar o desenvolvimento.
- **Cliente:** Pessoa ou organização que contrata e utiliza o sistema, sendo a principal fonte de requisitos e validação das entregas.
- **Iterativo:** Característica de processos que se repetem em ciclos, permitindo revisões e melhorias contínuas no desenvolvimento do sistema.
- **Documentação:** Registro formal das etapas, decisões e requisitos de um projeto, essencial para o acompanhamento e manutenção do sistema.

Referências Bibliográficas

SOMMERVILLE, Ian. Software Engineering. 6. ed. Harlow: Addison-Wesley, 2000.

ROYCE, Winston W. Managing the development of large software systems: concepts and techniques. In: PROCEEDINGS OF IEEE WESCON. Los Angeles: IEEE, 1970.

PUC. Métodos de desenvolvimento de produtos de interfaces gráficas digitais: da abordagem em cascata à ágil. Disponível em: https://www.maxwell.vrac.puc-rio.br/21839/21839_3.PDF. Acesso em: 14 set. 2026.

SOMMERVILLE, Ian. Engenharia de Software. 8. ed. São Paulo: Addison-Wesley, 2007.

MANIFESTO ÁGIL. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>. Acesso em: 14 set. 2026.